Instituto Profesional Virginio Gómez
Universidad de Concepción

Dominando el Comportamiento y la Memoria en C#

Semana 4: Métodos, Parámetros Especiales (ref, out) y Sobrecarga de Constructores

Asignatura: Lenguaje de Programación OO (0568)
Área de Informática – Instituto Profesional Virginio Gómez

Bloque 1: Invocación de Métodos y Mensajería

(Cátedra - 40 Minutos)

Comprendiendo cómo los objetos se comunican y ejecutan acciones dentro del paradigma de Programación Orientada a Objetos.

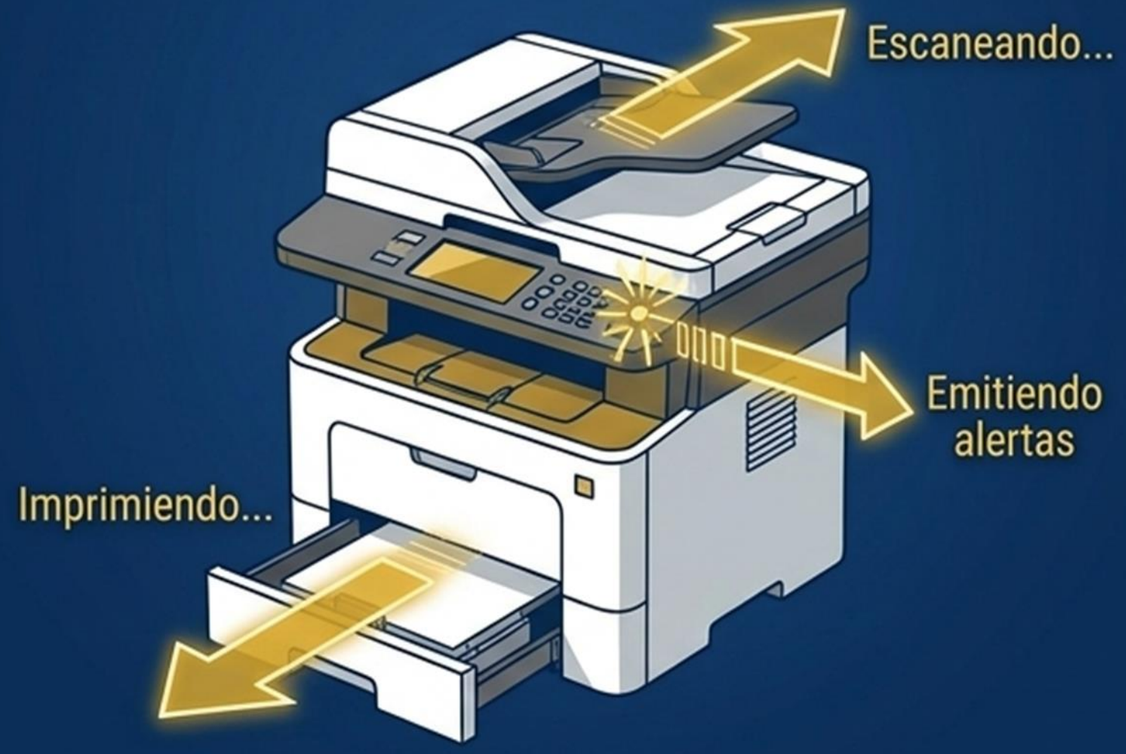


El Método Define el Comportamiento Activo del Objeto

Una máquina no es solo una colección de piezas de plástico y metal (sus atributos y campos). Su verdadero valor radica en lo que puede hacer.

En la Programación Orientada a Objetos, los objetos se envían mensajes entre sí invocando métodos. Un método es la encapsulación de una rutina de código que ejecuta una acción específica, transformando el estado interno del objeto o interactuando con el sistema externo.





El Operador Punto es el Botón de Acción

Para que un objeto ejecute uno de sus comportamientos, debemos presionar su botón correspondiente.



En C#, el operador de acceso a miembros (el punto **.**) es el mecanismo de mensajería. Conecta la referencia del objeto en memoria con la instrucción de ejecución del método.

Sin el punto, el objeto permanece inactivo; con el punto, el flujo del programa se desvía temporalmente para ejecutar la acción solicitada.

Anatomía y Firma de un Método en C#



public: Modificador de Acceso. Define quién puede presionar este botón (visibilidad).

void: Tipo de Retorno. Especifica qué devuelve la acción al finalizar (en este caso, nada).

CargarPapel: Identificador. El nombre de la acción. En C# y .NET, el estándar estricto es usar **PascalCase** (cada palabra inicia con mayúscula).

(int cantidadHojas): Parámetros. La información externa que la máquina necesita para operar. Forma parte integral de la firma del método.

Diferencia Técnica en los Flujos de Ejecución

`void`



Acciones de Estado (void):
Métodos que ejecutan rutinas internas, operan sobre los propios atributos de la clase o realizan salidas a consola. Al finalizar, el control vuelve al llamador, pero no se devuelve ningún dato en memoria.

`return`



Acciones de Retorno (return):
Métodos que procesan información y están obligados por su firma a emitir un tipo de dato específico de vuelta al llamador. La ejecución se detiene instantáneamente al alcanzar la instrucción return.

2

Bloque 2: Paso de Parámetros Avanzado en .NET

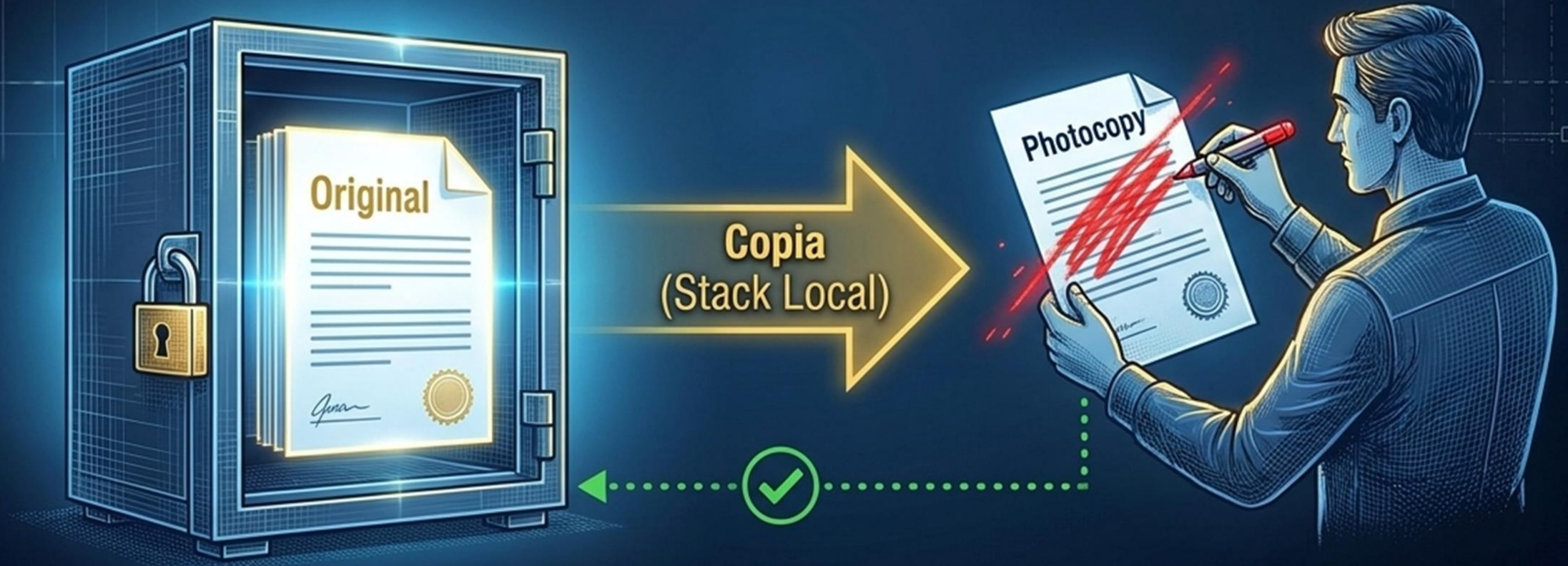
(Cátedra - 40 Minutos)

Mecanismos de memoria local (Stack) vs. memoria dinámica (Heap) mediante los modificadores ref y out.



El Comportamiento Estándar: Copia por Valor

Por defecto en C#, cuando pasamos un tipo de valor (como `int`, `decimal`, `bool`) a un método, el compilador realiza una copia exacta de esa información. Esta copia vive aislada en el Stack frame (la memoria temporal del método en ejecución). Cualquier cálculo, modificación o descuento que ocurra dentro del método afecta únicamente a la fotocopia. Cuando el método termina (retorna), el Stack frame se destruye y la copia se elimina. La variable original en el código llamador permanece inalterada.



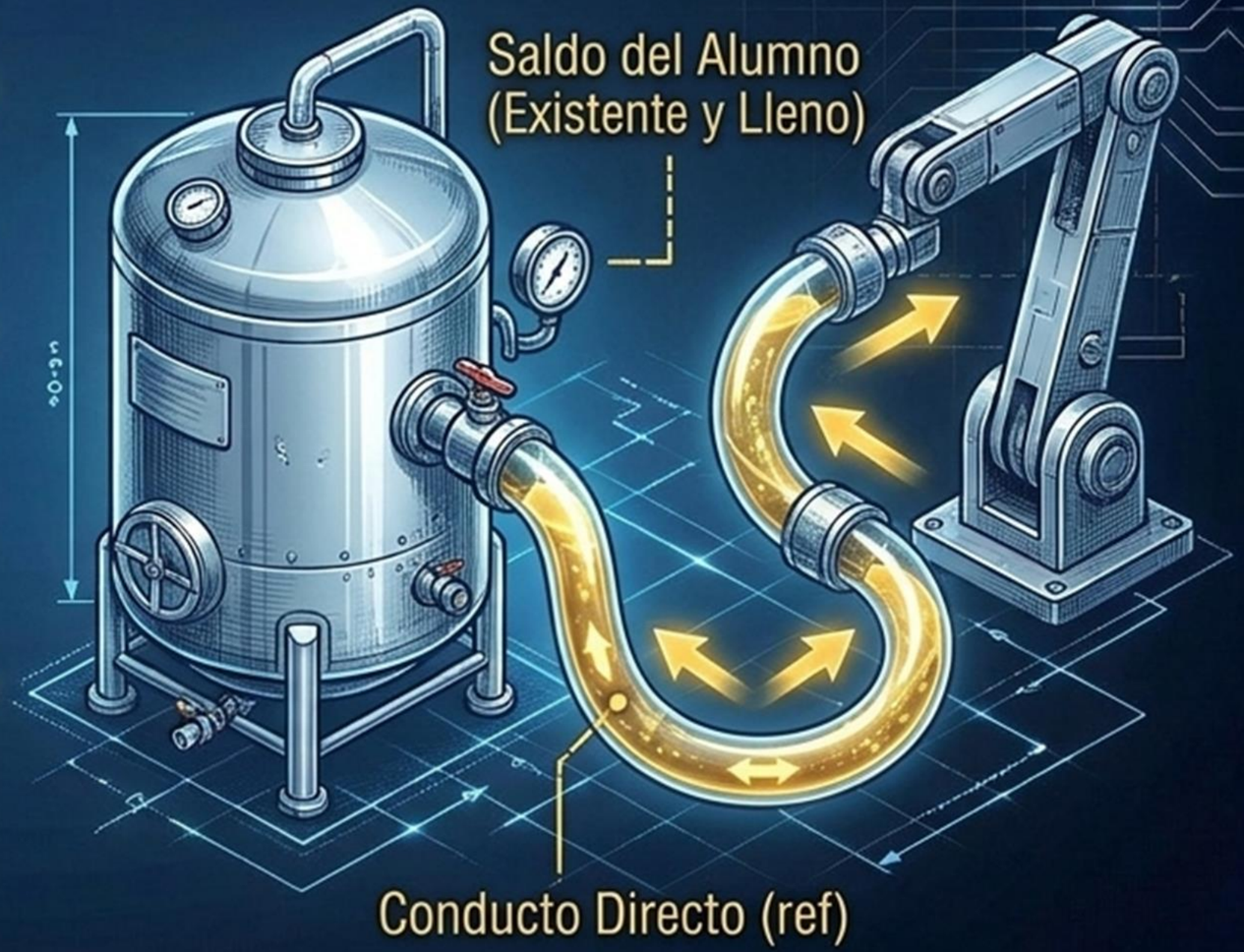
Referencia Bidireccional Directa (ref)

El modificador `ref` puentea el Stack local y crea un conducto directo hacia la ubicación original en memoria. No hay fotocopias.

1. Lectura y Escritura: El método puede leer el valor original y modificarlo directamente.

2. La Regla de Oro: Requiere inicialización previa y obligatoria.

3. Lógica de Dominio: El tanque de saldo de impresión debe existir en el mundo real antes de que el método ProcesarImpresion pueda conectarle una manguera para intentar descontar dinero. No se puede referenciar la nada.



Parámetro de Múltiple Salida (out)

El modificador **out** permite que un solo método devuelva múltiples valores simultáneamente, esquivando la limitación de un único **return**.

1. Solo Salida: Funciona como un receptáculo vacío entregado al método. No necesita, ni debe, tener información previa.

2. La Regla de Oro: Es una obligación absoluta del compilador. El método interno DEBE asignarle un valor a la variable **out** en todas las rutas lógicas antes de terminar.

3. Lógica de Dominio: La impresora está obligada a generar un comprobante (boleta con cantidad de páginas) desde cero al finalizar el proceso, y dejarlo en la bandeja de salida.



Matriz de Diagnóstico de Parámetros de .NET

Determina qué **modificador** utilizar basándote en los requisitos de memoria:

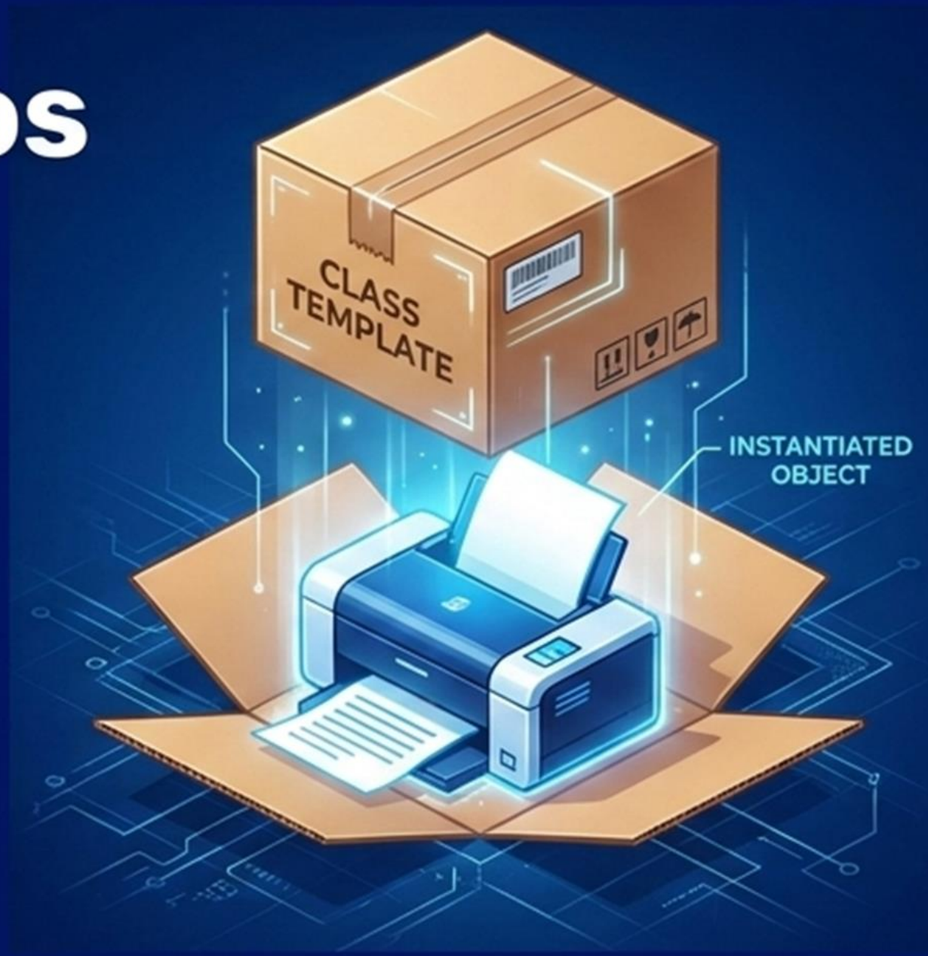
Modificador	¿Requiere Inicio?	Dirección de los Datos	Uso Común en Dominio IPVG
Paso por Valor (Por defecto)	Sí	Unidireccional (Entrada pura)	Pasar un número de documento para imprimir (solo lectura).
Referencia (**ref**)	Sí	Bidireccional (Lectura y Escritura)	Modificar un Saldo financiero existente.
Salida (**out**)	No (Se descarta si existe)	Unidireccional (Salida pura y obligatoria)	Emitir un nuevo comprobante de páginas gastadas tras la impresión.

13

Bloque 3: Métodos Constructores y Sobrecarga

(Cátedra - 40 Minutos)

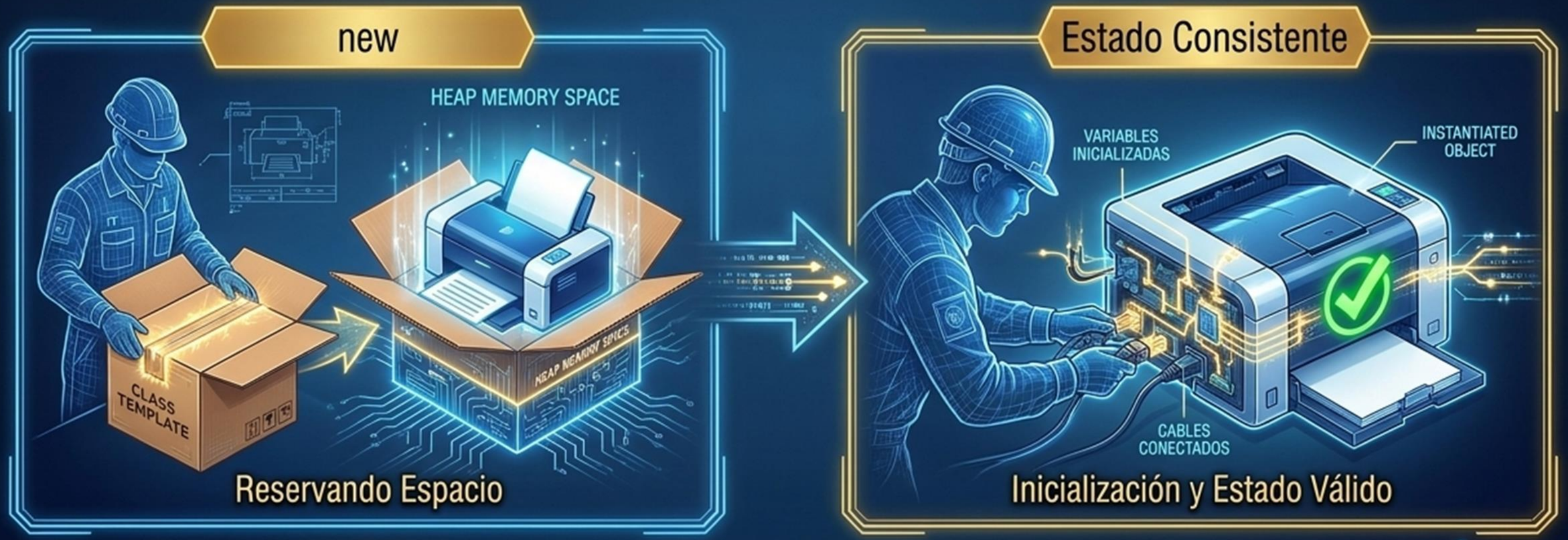
Reservando memoria en el Heap e inicializando estados consistentes en C#.



El Propósito Arquitectónico del Constructor

El constructor no es un método cualquiera; es la rutina de arranque obligatoria de un objeto.
Su propósito técnico es doble:

1. Actuar en conjunto con el operador new para reservar el espacio físico necesario en la memoria dinámica (Heap).
2. Garantizar un estado inicial consistente. Un objeto mal construido provocará NullReferenceException. El constructor asegura que todas las variables internas de instancia tengan valores válidos (cables conectados) antes de que el resto del sistema pueda interactuar con él.



Características Sintácticas Únicas en C#

Un constructor en C# rompe las reglas tradicionales de los métodos que acabamos de ver:

- Identidad Absoluta: Su nombre debe coincidir exactamente, respetando mayúsculas y minúsculas, con el nombre de la clase.
- Sin Retorno: No declara un tipo de retorno (ni siquiera void). Su función no es calcular salidas, sino orquestar el nacimiento del objeto mismo en memoria.

```
public class CuentaImpresion
{
    public CuentaImpresion(string rut)
    { // Reservar memoria en el Heap... }
}
```

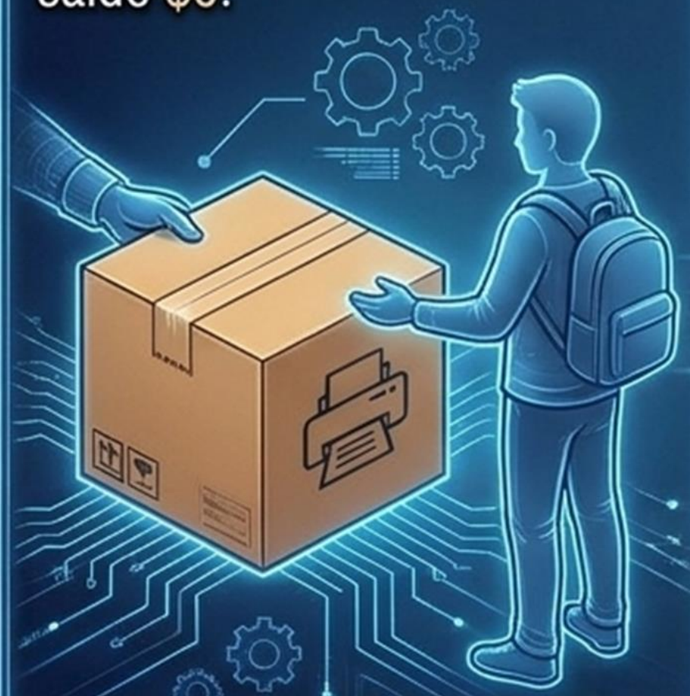
1 ¡Coincidencia exacta!

2 ¡Ningún tipo de retorno aquí!

Sobrecarga: Distintas Formas de Nacer

La Sobrecarga permite que una clase tenga múltiples constructores, siempre y cuando sus firmas (la cantidad y el tipo de los parámetros) sean distinguibles por el compilador. Esto permite crear objetos según la información disponible en ese instante exacto:

`CuentaImpresion()` → Crea una cuenta estándar con saldo $0.



`CuentaImpresion(decimal saldoBase)` → Crea una cuenta inyectando dinero previo.



`CuentaImpresion(decimal saldo, bool esPromo)` → Crea una cuenta aplicando lógicas de bonificación institucionales.



Encadenamiento y Delegación (this)

En el desarrollo profesional C#, duplicar lógica de inicialización en múltiples constructores sobrecargados viola el principio DRY (Don't Repeat Yourself).
Mediante la sintaxis : `this(...)`, un constructor puede delegar la carga de trabajo a otro constructor de la misma clase ANTES de ejecutar su propio bloque.
Esto canaliza toda la inicialización hacia un único Constructor Maestro, garantizando que las reglas de negocio base (como generar un ID único o auditar la creación) ocurran en un solo lugar blindado a errores.

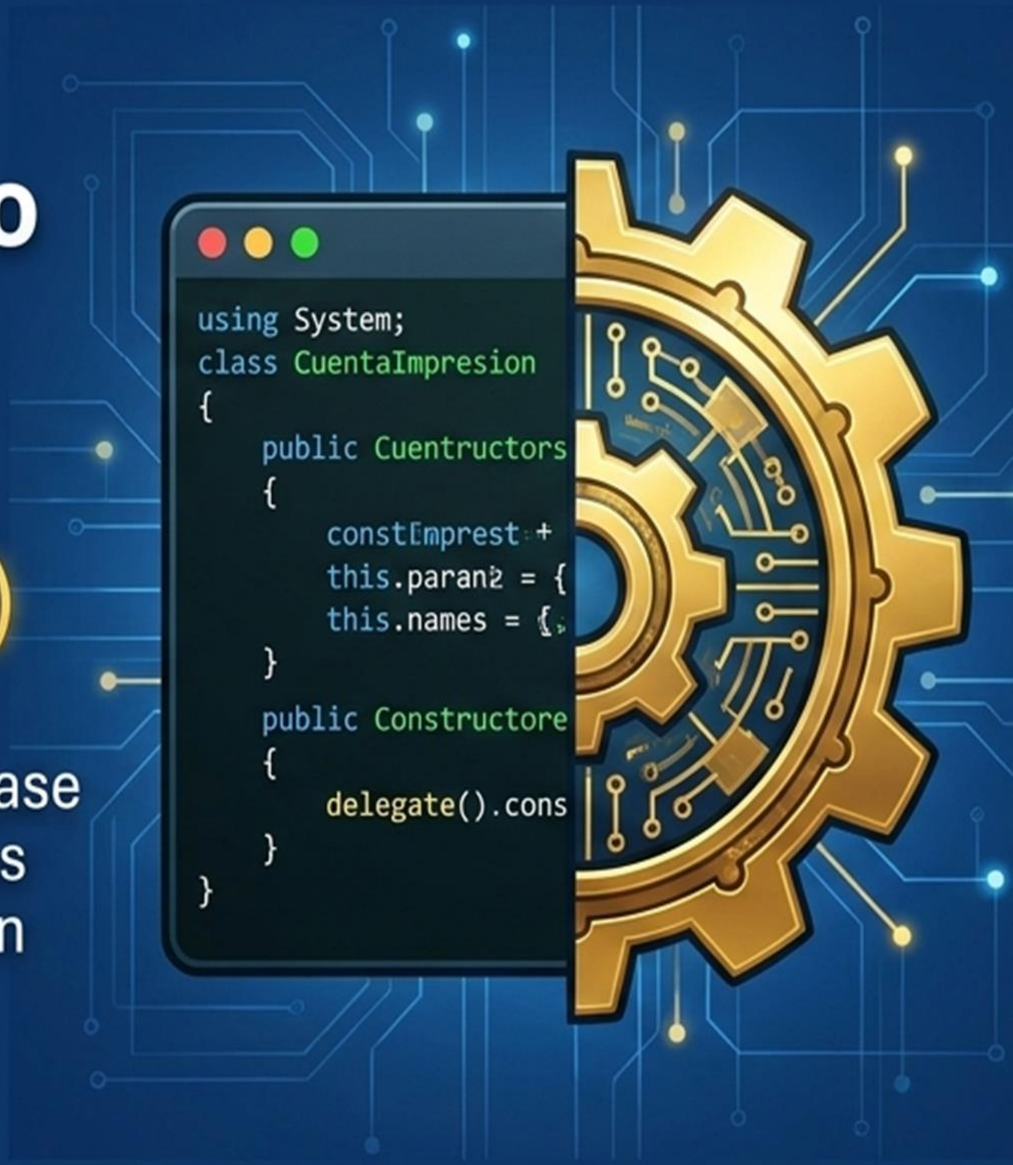


4

Bloque 4: Laboratorio Guiado - Sistema de Impresiones IPVG

(Laboratorio - 40 Minutos)

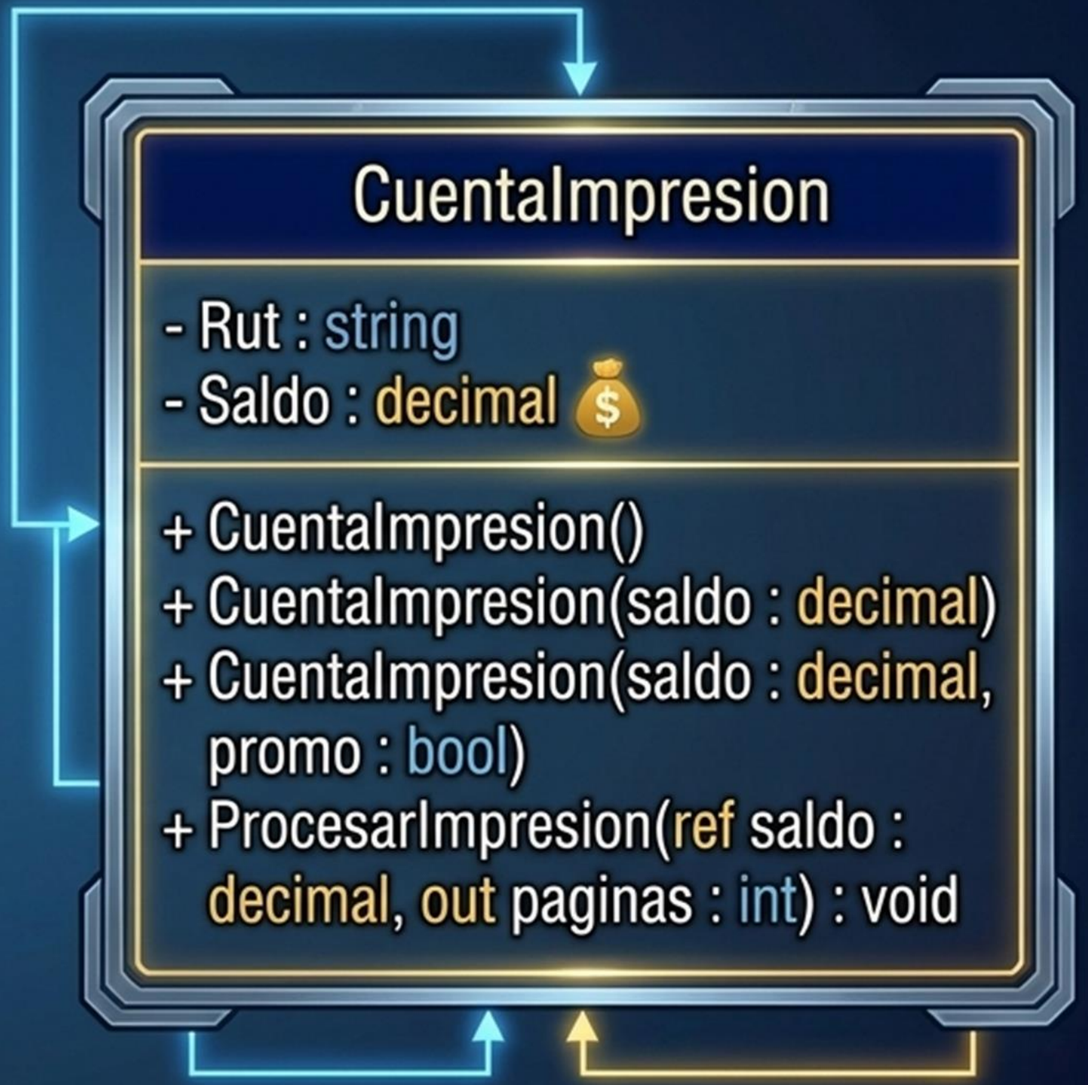
Aplicación práctica: Programando la clase CuentaImpresion, utilizando parámetros avanzados y constructores delegados en la consola.



Arquitectura del Sistema: El Reto de Código

Vamos a construir el módulo central del tarjetero de fotocopias de la sede:

1. Precisión Financiera: Usaremos el tipo de dato decimal (no double ni float) de dato decimal (no double ni float) que es el estándar de .NET para evitar pérdidas de precisión en operaciones de dinero real.
2. Flexibilidad de Inicio: Implementaremos 3 constructores (Por defecto, Saldo inicial, Promoción IPVG).
3. Transacción Robusta: El método central usará **ref** para modificar el fondo real del alumno y out para obligar a emitir un recibo con las páginas gastadas.



Código Paso 1: Implementando los Constructores

CuentaImpresion.cs | Program.cs

CuentaImpresion | CuentaImpresion - (CuentaImpresion) | saldo()

```
1   public class CuentaImpresion {
2       public decimal Saldo { get; private set; }
3
4       // Constructor Maestro
5       public CuentaImpresion(decimal saldoInicial) {
6           Saldo = saldoInicial;
7       }
8
9       // Constructor por defecto (Delega enviando 0)
10      public CuentaImpresion() : this(0.0m) {
10      }
11
12      // Constructor Promocional (Delega y luego aplica bono)
13      public CuentaImpresion(decimal saldo, bool promo) : this(saldo) {
14          if (promo) Saldo += 500.0m;
15      }
15  }
```

Código Paso 2: El Motor Transaccional (ref y out)

CuentaImpresion.cs

CuentaImpresion | ProcesarImpresion() | paginas

```
1   public void ProcesarImpresion(ref decimal saldoActual, int costoDoc, out int paginas)
2   {
3       // 1. OUT: Asignación obligatoria inicial (Bandeja vacía)
4       paginas = 0;
5
6       // 2. REF: Lectura del tanque original de dinero
7       if (saldoActual >= costoDoc)
8       {
9           // 3. REF: Modificación bidireccional directa
10          saldoActual -= costoDoc;
11
12          // 4. OUT: Emisión del dato de salida final
13          paginas = costoDoc / 20; // 20 pesos por hoja
14          Console.WriteLine($"Impresión OK. Saldo restante: ${saldoActual}");
15      } else {
16          Console.WriteLine("Saldo insuficiente para imprimir.");
17      }
    }
```

1 Lee el saldo ANTES del descuento y lo modifica en la fuente.

2 Asegura obligatoriamente que las páginas salgan calculadas.